

```

// Merged files from branch: main

// --- Start of pubspec.yaml ---
name: cl837_accelerometer
description: CL837 Accelerometer data reader
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.31.8
  permission_handler: ^11.0.1
  intl: ^0.18.0
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true
// --- End of pubspec.yaml ---

// --- Start of lib\accelerometer_service.dart ---
import 'dart:async';
import 'dart:core';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

enum AccelerometerFrequency {
  hz25(0, "25 Hz"),
  hz50(1, "50 Hz"),
  hz100(2, "100 Hz"),
  hz200(3, "200 Hz"),
  hz400(4, "400 Hz");

  final int value;
  final String label;
  const AccelerometerFrequency(this.value, this.label);
}

enum DataRateMode {
  normal(1, "Normal Rate"),
  doubleR(2, "2x Rate"),
  half(0.5, "1/2 Rate"),
  quarter(0.25, "1/4 Rate");

  final double factor;
  final String label;

```

```

    const DataRateMode(this.factor, this.label);
}

class AccelerometerData {
    final DateTime timestamp;
    final double x;
    final double y;
    final double z;
    final double frequency;
    final double interval;

    const AccelerometerData({
        required this.timestamp,
        required this.x,
        required this.y,
        required this.z,
        required this.frequency,
        required this.interval,
    });
}

class AccelerometerService {
    static const String _serviceUuid = 'aae28f00-71b5-42a1-8c3c-f9cf6ac969d0';
    static const String _dataCharacteristicUuid =
'aae28f01-71b5-42a1-8c3c-f9cf6ac969d0';
    static const String _txCharacteristicUuid =
'aae28f02-71b5-42a1-8c3c-f9cf6ac969d0';

    static const double _scaleFactor = 8.0 / 32768.0; // Per range ±8g

    final _dataStreamController = StreamController<AccelerometerData>.broadcast();
    final _stateStreamController = StreamController<bool>.broadcast();

    BluetoothCharacteristic? _dataCharacteristic;
    BluetoothCharacteristic? _txCharacteristic;
    StreamSubscription? _dataSubscription;

    DateTime? _lastDataTime;
    final List<double> _intervals = [];
    static const int _maxIntervals = 10;

    DataRateMode _currentMode = DataRateMode.normal;
    int _dataCounter = 0;
    AccelerometerData? _bufferedData;

    Stream<AccelerometerData> get dataStream => _dataStreamController.stream;
    Stream<bool> get connectionStream => _stateStreamController.stream;

    void setDataRateMode(DataRateMode mode) {
        _currentMode = mode;
        _dataCounter = 0;
        _bufferedData = null;
    }
}

```

```

void _processRawData(List<int> value) {
    if (value.length < 3) return;

    try {
        if (value[0] != 0xFF) return;

        final command = value[2];
        if (command == 0x0c) {
            final now = DateTime.now();

            // Calculate interval and frequency
            if (_lastDataTime != null) {
                final interval = now.difference(_lastDataTime!).inMicroseconds /
1000.0;
                _intervals.add(interval);
                if (_intervals.length > _maxIntervals) {
                    _intervals.removeAt(0);
                }
            }
            _lastDataTime = now;

            // Calculate average frequency
            double avgInterval = 0.0;
            if (_intervals.isNotEmpty) {
                avgInterval = _intervals.reduce((a, b) => a + b) / _intervals.length;
            }
            final double frequency = avgInterval > 0 ? (1000.0 / avgInterval) : 0.0;

            for (var i = 3; i < value.length - 1; i += 6) {
                if (i + 5 >= value.length) break;

                final rawX = _convertToSigned16(value[i] | (value[i + 1] << 8));
                final rawY = _convertToSigned16(value[i + 2] | (value[i + 3] << 8));
                final rawZ = _convertToSigned16(value[i + 4] | (value[i + 5] << 8));

                final x = rawX * _scaleFactor;
                final y = rawY * _scaleFactor;
                final z = rawZ * _scaleFactor;

                final data = AccelerometerData(
                    timestamp: now,
                    x: x,
                    y: y,
                    z: z,
                    frequency: frequency * _currentMode.factor, // Adjust reported
frequency
                    interval: avgInterval / _currentMode.factor // Adjust reported
interval
                );

                _processDataWithMode(data);
            }
        } catch (e) {

```

```

        debugPrint('Error processing data: $e');
    }
}

void _processDataWithMode(AccelerometerData data) {
    switch (_currentMode) {
        case DataRateMode.normal:
            _dataStreamController.add(data);
            break;

        case DataRateMode.doubleR:
            // Simulate double rate by interpolating between current and previous
data
            if (_bufferedData != null) {
                final interpolatedData = AccelerometerData(
                    timestamp: DateTime.fromMillisecondsSinceEpoch(
                        (_bufferedData!.timestamp.millisecondsSinceEpoch +
                            data.timestamp.millisecondsSinceEpoch) ~/ 2
                    ),
                    x: (_bufferedData!.x + data.x) / 2,
                    y: (_bufferedData!.y + data.y) / 2,
                    z: (_bufferedData!.z + data.z) / 2,
                    frequency: data.frequency,
                    interval: data.interval
                );
                _dataStreamController.add(_bufferedData!);
                _dataStreamController.add(interpolatedData);
                _dataStreamController.add(data);
                _bufferedData = null;
            } else {
                _bufferedData = data;
            }
            break;

        case DataRateMode.half:
            // Send every other sample
            if (_dataCounter % 2 == 0) {
                _dataStreamController.add(data);
            }
            _dataCounter++;
            break;

        case DataRateMode.quarter:
            // Send every fourth sample
            if (_dataCounter % 4 == 0) {
                _dataStreamController.add(data);
            }
            _dataCounter++;
            break;
    }
}

int _convertToSigned16(int value) {
    value &= 0xFFFF;
}

```

```

    return (value & 0x8000) != 0 ? -(0x10000 - value) : value;
}

Future<void> _setupNotifications(BluetoothCharacteristic characteristic) async
{
    try {
        await characteristic.setNotifyValue(false);
        await Future.delayed(const Duration(milliseconds: 100));

        final success = await characteristic.setNotifyValue(true);
        if (!success) {
            throw Exception('Could not enable notifications');
        }

        _dataSubscription = characteristic.value.listen(
            _processRawData,
            onError: (error) {
                debugPrint('Notification error: $error');
                _stateStreamController.add(false);
            },
        );

        _stateStreamController.add(true);
    } catch (e) {
        debugPrint('Setup notifications error: $e');
        _stateStreamController.add(false);
        rethrow;
    }
}

Future<void> start(BluetoothDevice device) async {
    try {
        final services = await device.discoverServices();

        final service = services.firstWhere(
            (s) => s.uuid.toString().toLowerCase() == _serviceUuid.toLowerCase(),
            orElse: () => throw Exception('Service not found'),
        );

        _dataCharacteristic = null;
        _txCharacteristic = null;

        for (var characteristic in service.characteristics) {
            final charUuid = characteristic.uuid.toString().toLowerCase();

            if (charUuid == _dataCharacteristicUuid.toLowerCase()) {
                _dataCharacteristic = characteristic;
                await _setupNotifications(characteristic);
            } else if (charUuid == _txCharacteristicUuid.toLowerCase()) {
                _txCharacteristic = characteristic;
            }
        }

        if (_dataCharacteristic == null || _txCharacteristic == null) {

```

```

        throw Exception('Required characteristics not found');
    }

    } catch (e) {
        debugPrint('Start error: $e');
        _stateStreamController.add(false);
        rethrow;
    }
}

Future<void> stop() async {
    await _dataSubscription?.cancel();
    _dataSubscription = null;
    _stateStreamController.add(false);
    _intervals.clear();
    _lastDataTime = null;
    _bufferedData = null;
    _dataCounter = 0;
}

void dispose() {
    stop();
    _dataStreamController.close();
    _stateStreamController.close();
}
}

// --- End of lib\accelerometer_service.dart ---

// --- Start of lib\main.dart ---
import 'dart:async';
import 'dart:collection';
import 'package:flutter/material.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:intl/intl.dart';
import 'accelerometer_service.dart';

void main() async {
    WidgetsFlutterBinding.ensureInitialized();
    FlutterBluePlus.setLogLevel(LogLevel.verbose, color: true);
    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({Key? key}) : super(key: key);

    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            debugShowCheckedModeBanner: false,
            title: 'CL837 Accelerometer',
            theme: ThemeData(
                primarySwatch: Colors.blue,
                visualDensity: VisualDensity.adaptivePlatformDensity,
            ),
        );
    }
}

```

```

    ),
    home: const BluetoothAccelerometerPage(),
  );
}
}

```

```

class OscilloscopePainter extends CustomPainter {
  final Queue<AccelerometerData> data;
  static const int maxPoints = 100;

  OscilloscopePainter(this.data);

  @override
  void paint(Canvas canvas, Size size) {
    final xPath = Path();
    final yPath = Path();
    final zPath = Path();

    final xPaint = Paint()
      ..color = Colors.red
      ..style = PaintingStyle.stroke
      ..strokeWidth = 2;

    final yPaint = Paint()
      ..color = Colors.green
      ..style = PaintingStyle.stroke
      ..strokeWidth = 2;

    final zPaint = Paint()
      ..color = Colors.blue
      ..style = PaintingStyle.stroke
      ..strokeWidth = 2;

    // Draw grid
    final gridPaint = Paint()
      ..color = Colors.grey.withOpacity(0.2)
      ..strokeWidth = 1;

    // Horizontal grid lines
    for (int i = 0; i <= 4; i++) {
      double y = size.height * i / 4;
      canvas.drawLine(Offset(0, y), Offset(size.width, y), gridPaint);
    }

    // Vertical grid lines
    for (int i = 0; i <= 8; i++) {
      double x = size.width * i / 8;
      canvas.drawLine(Offset(x, 0), Offset(x, size.height), gridPaint);
    }

    // Zero line
    final zeroPaint = Paint()
      ..color = Colors.grey.withOpacity(0.5)
      ..strokeWidth = 1;
  }
}

```

```

        canvas.drawLine(
            Offset(0, size.height / 2),
            Offset(size.width, size.height / 2),
            zeroPaint
        );

        if (data.isEmpty) return;

        var points = data.toList();
        for (var i = 0; i < points.length; i++) {
            var point = points[i];
            double x = size.width * i / maxPoints;

            // Normalize values from ±8g to 0..1
            double yX = size.height * (1 - (point.x + 8) / 16);
            double yY = size.height * (1 - (point.y + 8) / 16);
            double yZ = size.height * (1 - (point.z + 8) / 16);

            if (i == 0) {
                xPath.moveTo(x, yX);
                yPath.moveTo(x, yY);
                zPath.moveTo(x, yZ);
            } else {
                xPath.lineTo(x, yX);
                yPath.lineTo(x, yY);
                zPath.lineTo(x, yZ);
            }
        }

        canvas.drawPath(xPath, xPaint);
        canvas.drawPath(yPath, yPaint);
        canvas.drawPath(zPath, zPaint);
    }

    @override
    bool shouldRepaint(covariant CustomPainter oldDelegate) => true;
}

class BluetoothAccelerometerPage extends StatefulWidget {
    const BluetoothAccelerometerPage({Key? key}) : super(key: key);

    @override
    State<BluetoothAccelerometerPage> createState() =>
        _BluetoothAccelerometerPageState();
}

class _BluetoothAccelerometerPageState extends State<BluetoothAccelerometerPage>
{
    final AccelerometerService _accelerometerService = AccelerometerService();
    static const String targetDeviceName = 'CL837-0753644';

    BluetoothDevice? connectedDevice;
    AccelerometerData? latestData;
    bool isScanning = false;

```



```

bool isConnecting = false;

final Queue<AccelerometerData> plotData = Queue();
static const int maxPlotPoints = 100;

StreamSubscription? _dataSubscription;
DataRateMode _selectedDataRate = DataRateMode.normal;

@override
void initState() {
  super.initState();
  _initializeBluetooth();
}

Future<void> _initializeBluetooth() async {
  await _requestPermissions();
  await FlutterBluePlus.turnOn();
}

Future<void> _requestPermissions() async {
  final permissions = [
    Permission.bluetooth,
    Permission.bluetoothScan,
    Permission.bluetoothConnect,
    Permission.location,
  ];

  for (var permission in permissions) {
    await permission.request();
  }
}

Future<void> startScan() async {
  if (isScanning) return;

  setState(() {
    isScanning = true;
  });

  try {
    await FlutterBluePlus.startScan(timeout: const Duration(seconds: 5));

    FlutterBluePlus.scanResults.listen((results) {
      for (ScanResult r in results) {
        debugPrint('Found device: ${r.device.platformName}');
        if (r.device.platformName == targetDeviceName) {
          connectToDevice(r.device);
          FlutterBluePlus.stopScan();
          break;
        }
      }
    });

    await Future.delayed(const Duration(seconds: 5));
  }
}

```

```

        if (mounted) {
          setState(() {
            isScanning = false;
          });
        }
      } catch (e) {
        debugPrint('Error during scan: $e');
        if (mounted) {
          setState(() {
            isScanning = false;
          });
        }
        showError('Error during scan: $e');
      }
    }
  }

Future<void> connectToDevice(BluetoothDevice device) async {
  if (isConnecting) return;

  setState(() {
    isConnecting = true;
  });

  try {
    await device.connect(timeout: const Duration(seconds: 10));
    await _accelerometerService.start(device);

    _dataSubscription = _accelerometerService.dataStream.listen((data) {
      setState(() {
        latestData = data;
        plotData.add(data);
        if (plotData.length > maxPlotPoints) {
          plotData.removeFirst();
        }
      });
    });

    if (mounted) {
      setState(() {
        connectedDevice = device;
        isConnecting = false;
      });
    }
  } catch (e) {
    debugPrint('Error during connection: $e');
    if (mounted) {
      setState(() {
        isConnecting = false;
      });
    }
    showError('Error during connection: $e');
  }
}

```

```

Future<void> disconnectDevice() async {
  try {
    await _dataSubscription?.cancel();
    await _accelerometerService.stop();
    await connectedDevice?.disconnect();
  } catch (e) {
    debugPrint('Error during disconnect: $e');
  }

  if (mounted) {
    setState(() {
      connectedDevice = null;
      latestData = null;
      plotData.clear();
    });
  }
}

void showError(String message) {
  if (!mounted) return;

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
      backgroundColor: Colors.red,
      duration: const Duration(seconds: 3),
    ),
  );
}

Widget _buildDataRateDropdown() {
  return Card(
    elevation: 4,
    margin: const EdgeInsets.symmetric(horizontal: 16.0),
    child: Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16.0),
      child: DropdownButton<DataRateMode>(
        value: _selectedDataRate,
        isExpanded: true,
        underline: Container(),
        items: DataRateMode.values.map((mode) {
          return DropdownMenuItem<DataRateMode>(
            value: mode,
            child: Text(mode.label),
          );
        }).toList(),
        onChanged: (DataRateMode? newMode) {
          if (newMode != null) {
            setState(() {
              _selectedDataRate = newMode;
              _accelerometerService.setDataRateMode(newMode);
            });
          }
        }
      )
    )
  );
}

```

```

    },
  ),
),
);
}

```

```

Widget _buildCurrentValuesCard() {
  return Card(
    elevation: 4,
    margin: const EdgeInsets.all(16.0),
    child: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              const Text(
                'Accelerometer Data',
                style: TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
              if (latestData != null)
                Column(
                  crossAxisAlignment: CrossAxisAlignment.end,
                  children: [
                    Text(
DateFormat('HH:mm:ss.SSS').format(latestData!.timestamp),
                      style: TextStyle(
                        color: Colors.grey[600],
                        fontSize: 14,
                      ),
                    ),
                    Text(
                      'Rate: ${latestData!.frequency.toStringAsFixed(1)} Hz',
                      style: TextStyle(
                        color: Colors.grey[600],
                        fontSize: 14,
                      ),
                    ),
                    Text(
                      'Interval: ${latestData!.interval.toStringAsFixed(1)}
ms',
                      style: TextStyle(
                        color: Colors.grey[600],
                        fontSize: 14,
                      ),
                    ),
                  ],
                ),
            ],
          ),
        ],
      ),
    ),
  ),
);
}

```

```

    ],
  ),
  const SizedBox(height: 16),
  if (latestData != null) ...[
    _buildAxisRow('X', latestData!.x, Colors.red),
    const SizedBox(height: 8),
    _buildAxisRow('Y', latestData!.y, Colors.green),
    const SizedBox(height: 8),
    _buildAxisRow('Z', latestData!.z, Colors.blue),
    const SizedBox(height: 16),
    Container(
      height: 200,
      decoration: BoxDecoration(
        border: Border.all(color: Colors.grey.withOpacity(0.2)),
        borderRadius: BorderRadius.circular(8),
      ),
      child: ClipRRect(
        borderRadius: BorderRadius.circular(8),
        child: CustomPaint(
          painter: OscilloscopePainter(plotData),
          size: const Size(double.infinity, 200),
        ),
      ),
    ),
  ],
] else
  const Center(
    child: Text(
      'Waiting for data...',
      style: TextStyle(
        fontSize: 16,
        color: Colors.grey,
      ),
    ),
  ),
),
],
),
);
}

```

```

Widget _buildAxisRow(String axis, double value, Color color) {
  return Row(
    children: [
      SizedBox(
        width: 30,
        child: Text(
          axis,
          style: TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
            color: color,
          ),
        ),
      ),
    ],
  ),
);

```

```

Expanded(
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      LinearProgressIndicator(
        value: (value + 8.0) / 16.0, // Normalize value from -8g to +8g
        backgroundColor: color.withOpacity(0.1),
        valueColor: AlwaysStoppedAnimation<Color>(color),
      ),
      const SizedBox(height: 4),
      Text(
        '${value.toStringAsFixed(3)} g',
        style: const TextStyle(
          fontFamily: 'Monospace',
          fontSize: 14,
        ),
      ),
    ],
  ),
),
],
);
}

```

```

@override
void dispose() {
  _dataSubscription?.cancel();
  _accelerometerService.dispose();
  disconnectDevice();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('CL837 Accelerometer'),
      actions: [
        if (connectedDevice != null)
          IconButton(
            icon: const Icon(Icons.bluetooth_connected),
            tooltip: 'Disconnect',
            onPressed: disconnectDevice,
          ),
      ],
    ),
    body: Column(
      children: [
        Container(
          color: Colors.grey[100],
          padding: const EdgeInsets.all(16.0),
          child: Row(
            children: [
              Expanded(

```

```

        child: ElevatedButton.icon(
          icon: const Icon(Icons.search),
          label: Text(
            isScanning ? 'Scanning...' :
            isConnecting ? 'Connecting...' :
            'Scan for Device'
          ),
          onPressed: (isScanning || isConnecting) ? null : startScan,
        ),
      ),
    ],
  ),
),
if (connectedDevice != null) ...[
  _buildDataRateDropdown(),
  Expanded(
    child: SingleChildScrollView(
      child: _buildCurrentValuesCard(),
    ),
  ),
],
),
);
}
}
// --- End of lib\main.dart ---

```